

FUNDAMENTOS DE PROGRAMAÇÃO



Projeto de Fundamentos de Programação

Documentação técnica

GRUPO N°29

107142	<i>Nuno Aguilar</i> <i>Nunoaguilar6@tecnico.ulisboa.pt</i>
107143	<i>Madalena Filipe</i> <i>Madalena.filipe@tecnico.ulisboa.pt</i>

1

Licenciatura em Engenharia Mecânica

Licenciatura em Engenharia e Arquitetura Naval

Lisboa, Mês de Ano

Índice

1.	Documentação técnica.....	3
1.1.	Arquitetura do programa.....	3
1.2.	Projeto detalhado	5
1.3.	Módulo Nome do Módulo	13

1. Documentação técnica

1.1. Arquitetura do programa

O presente programa desempenha um robô que se move omnidirecionalmente numa janela definida com o objetivo de realizar a sua limpeza. O robô é responsável por garantir a higiene de um restaurante, deslocando-se até aos pontos de maior sujidade e desviando-se dos obstáculos presentes no seu percurso. O desempenho do robô, a localização dos obstáculos e das zonas de sujidade dependem da implementação escolhida, tornando-se, o programa, mais complexo em cada variação de implementação. Existem, no total, 3 implementações diferentes, no entanto, a última subdivide-se em dois métodos de disposição do cenário diferentes que, por sua vez, subdividem-se também em duas opções de localização de pontos de sujidade distintas. O programa funciona através de um ficheiro (Projeto.py) que engloba as quatro principais funções, referentes às diversas implementações (a terceira implementação incorpora duas funções distintas) e outras onze funções que calculam o vetor unitário e a norma do vetor, definem os vários movimentos do robô (para exercer a limpeza, dirigir-se à zona de carregamento, eleger a zona de carregamento mais próxima), criam os botões, determinam os pontos inválidos, exibem os menus e geram obstáculos aleatórios para a terceira implementação. O programa contém também mais dez ficheiros referentes às várias classes do robô, sujidade, obstáculos, botões e menus. Inclui também dois ficheiros de texto (.txt), que contêm a informação das coordenadas dos obstáculos e dos pontos de sujidade para a terceira implementação.

3

Primeira implementação: A implementação mais básica apresenta, na sua janela gráfica, apenas um obstáculo, uma mesa circular no centro que deverá ser contornada, gerada pela Classe Table, e duas estações de carregamento nos extremos, geradas pela Classe Dock. Para além das referidas foram também importadas as classes Waiter (representação do robô) e Mess (representação da sujidade). As funções criadas para esta implementação foram a vetor, Limpeza, choosedock, invalidlocal, exitbutton, e exitclicked (que serão explicadas posteriormente). Foram utilizadas também as funções move e cont. (do módulo Table).

FUNDAMENTOS DE PROGRAMAÇÃO

Segunda implementação: Nesta implementação acrescentámos outros quatro obstáculos: duas cadeiras (Classe Chair), uma planta (Classe Plant) e um aquário (Classe FishTank). Para além disso foi adicionada ao robô uma bateria com capacidade de percorrer duas vezes a área de navegação, que descarrega de acordo com a distância percorrida pelo robô. Com este acréscimo, para além das funções referidas anteriormente, foram criadas novas funções como capacity, battery, batteryanimation, checkbat, gocharge.

Terceira implementação: Nesta implementação criámos duas funções principais para diferenciar os dois métodos de posicionamento dos obstáculos: Imp3File e Aleatger. No método de gerar os obstáculos através de um ficheiro, o programa vai buscar os dados ao ficheiro Sala.txt. Para o método de gerar aleatoriamente foi criada uma nova função, aleatpoint. Surgem, dentro destas ainda outras duas opções relativas ao posicionamento dos pontos de sujidade, podendo estes serem estipulados através de cliques de rato, ou através da leitura de um ficheiro de texto, Limpeza.txt. A restante dinâmica desta implementação é semelhante à da anterior.

4

A programação do movimento do robô difere da primeira implementação para a segunda/terceira dado que a partir da implementação intermediária, o robô passa a ter uma rota definida.

Na primeira implementação é utilizada a função vetor() para calcular o vetor que representa a direção e distância entre a posição do robô e a posição da sujidade. De seguida, o robô move-se em vetores unitários de cada vez, enquanto a norma do vetor for maior que 1, de modo a que se dirija gradualmente ao encontro da sujidade. A cada passo, a norma do vetor é verificada para garantir que ainda haja um caminho significativo a percorrer, quando esta é menor ou igual a 1, o robô está próximo o suficiente da sujidade e não precisa se mover mais nessa direção.

O processo de movimentação do robô é repetido até que a norma do vetor seja menor ou igual a 1, o que indica que o robô está próximo o suficiente da sujidade e não precisa se mover mais nessa direção. A cada passo, o vetor é recalculado para levar em consideração a nova posição do robô.

Para o movimento da segunda/terceira implementação é iniciado um loop (for i in range()), onde o robô inicia o seu trajeto horizontal. Durante este movimento, são executadas as funções searchmess() para verificar se existem pontos de sujidade a limpar, e cont() para garantir que o robô se desvia dos obstáculos. Quando atinge o x

FUNDAMENTOS DE PROGRAMAÇÃO

definido (encontra-se perto do limite lateral da janela) o robô inicia o seu movimento vertical. Durante este movimento é chamada a função `gocharge` para direcionar o robô em direção à estação de carregamento mais próxima caso este tenha uma carga inferior a 15% da capacidade. Se não for o caso, ao atingir o valor da componente `y` estipulado retoma o movimento horizontal e repete-se o processo. Quando o loop termina, o robô encontra-se na mesma reta horizontal (porém) lado oposto, que a dock e move-se até lá.

1.2. Projeto detalhado

1.2.1 Classe Waiter - Esta classe cria e desenha graficamente o robô e incorpora funcionalidades do mesmo. Nela estão inseridos métodos gráficos, relativos ao seu movimento e métodos associados à bateria.

- Método `__init__()`, o robô é criado
- Método `move(dx, dy)` - move o robô na direção especificada pelos valores `dx` e `dy` e desconta a bateria com o movimento se esta estiver ativa.
- Método `getX()`, `getY()`, `getCenter()` e `getRadius()` - retornam as coordenadas `x` e `y` do centro do círculo, o próprio centro e o raio, respectivamente.
- Método `draw(win)`, `undraw()`, `drawbattery(win)`, `undrawbattery()` - desenharam e eliminam todos os objetos gráficos associados ao robô/bateria
- Método `capacity(capacity)` - define a capacidade total da bateria
- Método `battery(charge)` ativa a funcionalidade da bateria e define o seu valor de carga inicial
- Método `Batteryanimation(charge)` cria uma animação de mudança de cor na bateria para representar o nível de carga.
- Método `checkbat()` - verifica se o nível de carga da bateria está abaixo de 15% da capacidade total, para, caso esteja, o robô retorne à dock.

Classe Mess

Esta classe cria e desenha graficamente a sujidade e incorpora funcionalidades da mesma. Este objeto é definido pelas variáveis âncora (ponto de referência central) e `size`.

FUNDAMENTOS DE PROGRAMAÇÃO

- Método "draw" e "undraw" - responsáveis por desenhar e apagar todos os objetos gráficos

-Método retornaAncoraX, retorna Ancora Y, getCenter, getmess devolvem ao utilizador as coordenadas x e y da âncora e o tamanho do ponto de sujidade.

-Método move - Move todos os elementos gráficos associados ao ponto de sujidade na direção especificada pelos valores dx e dy.

Classe Dock - - Esta classe cria e desenha graficamente a zona de carregamento.

- Método draw - responsável por desenhar as docks

- Método getX, getY, getWidth, getHeight, getXY - devolvem ao utilizador as coordenadas x e y da âncora e as dimensões das docks.

Classe Table, Plant, Fishtank, Chair

6

Estas classes representam os obstáculos, criando os objetos (mesa, planta, aquário, cadeira) no ambiente gráfico. Estes objetos são definidos pelas variáveis âncora e size e também definem uma caixa delimitadora (hitbox) para deteção de colisão.

- Método "draw" e "undraw" - responsáveis por desenhar e apagar todos os objetos gráficos

-Método retornaAncoraX, retorna Ancora Y, get(variável) devolvem ao utilizador as coordenadas x e y da âncora e o tamanho dos obstáculos.

- Método cont(hX, hY) – verifica se as coordenadas hX e hY, (coordenada do x e do y do robô no ficheiro principal) estão em contacto com o obstáculo ao verificar a condição “if self.xmin < hX < self.xmax and self.ymin < hY < self.ymax “ (xmin ,xmax , ymin, ymax representam as coordenadas dos limites das hitboxes dos obstáculos) e retorna um valor booleano indicando se houve contacto. Caso retorne True executa condições que dependem da implementação:

1ªimplementação: Se o contacto ocorrer nas bordas laterais da hitbox, o robô move-se para cima ou para baixo, dependendo da posição do ponto clicado em relação ao robô. Se ocorrer um contato na borda superior ou inferior da hitbox, o robô move-

FUNDAMENTOS DE PROGRAMAÇÃO

se para a direita ou para a esquerda com base na posição do ponto clicado. Essas ações permitem que o robô evite obstáculos e siga em direção ao ponto desejado.

2ª e 3ª implementação: Nas implementações em que o robô tem uma rota definida difere. Se o contacto estiver mais perto da componente y mínima da hitbox do que da componente y máxima, o robô move para baixo e vice-versa. De seguida executa o seu movimento horizontal normal até atingir o canto seguinte da hitbox . Quando o atinge, retoma até ao valor componente y que se encontrava antes da colisão e retoma ao trajeto definido.

Classes Menu e SubMenu – Estas classes criam um menu gráfico que permite ao utilizador seleccionar a implementação desejada.

- **Método clicar**- responsável por criar um loop onde o programa aguarda o clique do utilizador. Ele verifica qual botão foi clicado e retorna um valor correspondente à opção seleccionada.

Método main – cria uma instância da classe "Menu"/"Submenu" e chama o método clicar() para exibir o menu e obter a escolha do usuário.

7

Classes Button e MenuButton –

Estas classes criam, desenhavam graficamente e permitem a interação de botões do programa.

- **Método clicked** - Verifica se o botão foi clicado. Retorna True se o botão estiver ativo e se as coordenadas do ponto p estiverem dentro dos limites do retângulo do botão.

- **Método getLabel** -Retorna o texto do botão.

- **Método activate** -Ativa o botão

- **Método deactivate** -Desativa o botão

Programa Principal

Este módulo contém as nossas funções principais, relativas às implementações, e outras funções e classes que desempenham o programa. A estrutura neste módulo está

FUNDAMENTOS DE PROGRAMAÇÃO

organizada através de variáveis booleanas e blocos de código condicionais de modo a garantir uma flexibilidade de adaptação às diferentes exigências de cada implementação. À medida que o programa é executado, ele verifica o estado dessas variáveis booleanas e executa o bloco de código correspondente, alterando assim o comportamento do programa.

Primeiramente importamos os módulos necessários, como "graphics", "time", "math" e "random", e também as várias classes do projeto, como "Menu", "SubMenu", "Button", "Waiter", "Mess", "Dock", "Table", "FishTank", "Chair" e "Plant".

De seguida, definimos um conjunto de funções que para que fossem utilizadas em cada uma das implementações. Cada uma contém diversas entradas permitindo que estas sejam ajustadas de acordo com a diferença de requisitos e necessidades na variação das implementações, não comprometendo a fluência e bom funcionamento do programa.

Funções:

8 - **vetor (aX, aY, bX, bY)** - Calcula o vetor unitário entre dois pontos no plano.

Retorna uma lista que contém as componentes x,y e a norma do vetor unitário.

- **exitbutton (win, ancora, width, height)** - Cria e retorna um botão de saída com a posição, largura, altura e cor especificadas.

- **exitclicked (exitButton, p, win)** - Verifica se o botão de saída foi clicado e fecha a janela do programa.

- **invalidlocal (obstacles, clique, wradius, width, height)** - Verifica se o ponto clicado é inválido, com base na lista de obstáculos e nas dimensões da janela. Retorna "True" se o clique for inválido, "False" caso contrário.

- **choosedock (waiter,dock1,dock2)** - Escolhe o local do dock mais próximo para o robô regressar. Retorna as coordenadas (x, y) do centro da doca escolhida.

- **Limpeza (waiter, wradius, l, speed, move)** - Realiza uma animação que simula o movimento de limpeza do robô até uma posição específica.

- **gocharge (dock1,dock2,waiter,speed,charge,traject)** - Move o robô até à dock mais conveniente para recarregar sua bateria.

FUNDAMENTOS DE PROGRAMAÇÃO

- **searchmess (sujeidades, waiter, traject, reach, wradius, obstaculos, update, move)**- procura sujeidades na sala para realizar a sua limpeza. Esta função percorre a lista de sujeidades e verifica se alguma está dentro do alcance vertical e aproximadamente na mesma posição horizontal do robô de limpeza. Se uma sujeidade é encontrada, o robô move-se em direção a ela, realiza a limpeza e atualiza a posição da sujeidade para fora da janela. Em seguida, o robô retorna ao trajeto de limpeza da sala, evitando obstáculos pelo caminho.

- **menu()** - projeta o menu e executa as ações correspondentes às opções escolhidas.

- **filemenu()** - projeta o submenu de opções do modo "ficheiro" e executa as ações correspondentes às opções escolhidas.

- **aleatmenu ()** - projeta o submenu de opções do modo "aleatório" e executa as ações correspondentes às opções escolhidas.

- **aleatpoint (win, wradius, width, height, buttonwidth, buttonheight)** - Esta função gera obstáculos aleatoriamente dispersos pela sala. São geradas cinco coordenadas aleatórias distintas dentro da área disponível na janela. Para cada coordenada, um tipo de forma é escolhido aleatoriamente (fishtank, table, chair, plant) o seu objeto correspondente é criado. No final, retorna uma lista de objetos aleatórios com suas localizações definidas também aleatoriamente na janela para a terceira implementação.

O bloco inicial do código de cada implementação destina-se à criação, desenho e definição das dimensões e cores da janela gráfica, [GraphWin()], das docks (Dock), dos obstáculos (Table, Plant, FishTank, Chair), do robô (Waiter) e dos botões(Button).

Primeira Implementação:

-**Imp1()** : Esta função representa a implementação básica do projeto. Nesta implementação existe um único obstáculo do qual o robô terá de se desviar, uma mesa circular (Table). Nela é iniciado um loop (while True) onde o utilizador clica num ponto da janela à escolha. O programa verifica se o clique é válido, [invalidlocal()] e, caso não seja, aparece um aviso de erro, . Quando é válido, a sujeidade (Mess) é criada e desenhada nesse ponto e o robô desloca-se até lá e realiza a sua limpeza, [Limpeza()]. Em seguida, o programa escolhe a dock mais próxima, [choosedock()] para o robô regressar [vetor()]. Durante todo o movimento do robô, é verificado se o robô vai contra o obstáculo, [cont()] da classe Table]. O loop continua indefinidamente até que o programa seja encerrado. Para além disso, utilizámos os métodos

FUNDAMENTOS DE PROGRAMAÇÃO

`sleep()` do módulo `time` e o método `uptade` para que o robô faça uma pequena pausa e para que fosse ajustada a sua velocidade.

Segunda Implementação

-`Imp2()` : Na implementação intermediária são agora criados e desenhados 5 obstáculos, duas poltronas (`Chair`), uma mesa (`Table`) , um aquário (`FishTank`) e uma planta (`Plant`). Para facilitar a definição e manipulação do movimento do robô a desviar-se de todos os obstáculos, agrupámo-los numa lista, `group`:

```
group = []
```

```
for g in [table,fishtank,chair1,chair2,plant]:
```

```
    group.append(g)
```

Foi também criada uma lista destinada aos pontos de sujidade (`sujidade[]`).

Nesta implementação o utilizador pode escolher vários pontos de sujidade pelo que o programa aguarda a interação na janela. Primeiramente, há a verificação que o clique foi dentro dos limites estabelecidos e, caso tenha sido, o programa verifica se o clique foi num dos botões : Se o botão "Start" tiver sido pressionado e existir pelo menos um ponto de sujidade no ambiente o programa inicia o seu movimento.

```
if startbutton.clicked(p) and len(sujidades)>0:
```

```
(...)
```

```
    pointchosen = True
```

Caso não haja pontos seleccionados, o programa aguarda o utilizador escolher um ponto dentro da janela e dentro dos limites definidos para adicionar o ponto de sujidade que posteriormente é adicionado à lista `sujidades` esta é desenhada na janela.

```
sujidades.append(Mess(p,30))
```

```
sujidades[len(sujidades)-1].draw(win)
```

O programa verifica, se o robô, no seu movimento horizontal, está numa zona que contém algum ponto de sujidade. Se estiver, ele elimina-a e atualiza a lista de `sujidades`.

FUNDAMENTOS DE PROGRAMAÇÃO

Nesta implementação é também adicionada uma bateria através dos métodos `capacity(capacity)`, `battery(charge)`, `Batteryanimation(charge)` e `checkbat()` explicados anteriormente.

Terceira Implementação: Na terceira implementação existem duas opções de disposição dos obstáculos na sala: lidas através de um ficheiro (`Sala.txt`) ou geradas aleatoriamente. Assim, optámos por separar esta implementação por duas funções principais diferentes : `Imp3File(click, file )` e `Aleatger(click ,file)`. Também a localização dos pontos de sujidade pode ser feita de duas formas diferentes, daí as duas entradas em cada função, através de cliques (`click`), à semelhança da implementação anterior, ou lidas através de um ficheiro (`Limpeza.txt`), que permitiu que não houvesse grandes alterações no código ao utilizar as condições “if click:” ou “if file:”.

- Imp3File

Para esta implementação foi alterado o método do bloco inicial do código que define as dimensões da janela, obstáculos e docks, sendo que as coordenadas do ponto que definem estes objetos e as dimensões são dadas por um ficheiro de texto, utilizando o método `readlines()`.

-Aleatger

Nesta implementação a localização dos obstáculos e as formas dos mesmos ao gerados aleatoriamente. Para isso criámos a função `aleatpoint()`, explicada anteriormente, que utiliza o método `randrange` da biblioteca `random`. Para garantir que não houvesse sobreposição de obstáculos, limitámos a área de pontos possíveis.

```
randomx = randrange(2*wradius + 60 + wradius, width - 2*wradius - 60 -  
wradius, 2*60 + 2*wradius + 7)
```

```
randomy = randrange(2*wradius + 60 + wradius, height - 2*wradius - 60,  
2*wradius + 2*60 + 2*wradius)
```

A primeira e segunda entrada representam o início e fim, respectivamente, de onde os objetos podem ser desenhados, garantindo que o robô consiga percorrer sua rota

FUNDAMENTOS DE PROGRAMAÇÃO

normalmente. A terceira entrada define um intervalo de espaçamento entre os pontos possíveis de serem desenhados, evitando assim sobreposições.

Variáveis :

aX, aY, bX, bY (float) : representam as coordenadas das componentes x e y de dois pontos.

win (GraphWin): Janela gráfica onde o botão será desenhado.

ancora (Point): Ponto central do objeto.

width (int) : Largura.

Height (int): Altura.

exitButton (Button) : Botão de saída a ser verificado.

p (Point): Ponto clicado pelo utilizador.

obstacles (list): Lista de objetos que representam os obstáculos.

clique (Point): Ponto que representa o clique feito pelo utilizador.

wradius (float): O raio do objeto Waiter.

waiter (Waiter): Objeto Waiter (robô).

dock1 (Dock): Primeira doca.

dock2 (Dock): Segunda doca.

l (Point): Objeto Point representando o centro do objeto alvo da limpeza.

speed (float): Velocidade de atualização da animação.

move (float): Valor do deslocamento para cada movimento.

sujidades (list): Lista de sujidades presentes na sala.

traject (float): Posição y do trajeto de limpeza na sala.

reach (float): Alcance vertical máximo para detecção de sujidades.

buttonwidth (int): Largura do botão.

buttonheight (int): Altura do botão.

FUNDAMENTOS DE PROGRAMAÇÃO

1.3. Módulo Waiter

Tipo: Classe

Função: A classe "Waiter" é responsável por criar e desenhar graficamente o robô na janela. Ela possui métodos para para movimentar o robô nas coordenadas x e y e calcular a sua posição no momento. Além disso, a classe também mede a carga da bateria do robô e calcula a quantidade necessária para que ele possa se deslocar até a Dockstation e recarregar. Além disso, a cor da bateria é alterada de acordo com a sua carga, permitindo uma animação visual ao durante o descarregamento e carregamento.

1.3.1. Parâmetros

centro, raio, win, dx, dy, capacity, charge

1.3.2. Métodos

init, move, getX, getY, getCenter, , getRadius draw, undraw, drawbattery, undrawbattery, batteryanimation, battery, capacity, checkbat

13

Table

Tipo: Classe

Função: Esta classe cria e desenha graficamente uma mesa circular.

1.3.3. Parâmetros

ancora, size, hb, hX, hY, win

1.3.4. Métodos (aplicável para classes)

__init__, cont, draw, undraw, retornaAncoraX, retornaAncoraY, gettable

1.4. Plant

Tipo: Classe

FUNDAMENTOS DE PROGRAMAÇÃO

Função: Esta classe cria e desenha graficamente uma planta.

1.4.1. Parâmetros

ancora, size, hb, hX, hY, win

1.4.2. Métodos

__init__, cont, draw, undraw, retornaAncoraX, retornaAncoraY, getplant

1.5. Chair

Tipo: Classe

Função: Esta classe cria e desenha graficamente uma poltrona.

1.5.1. Parâmetros

ancora, size, hb, hX, hY, win

1.5.2. Métodos

__init__, cont, draw, undraw, retornaAncoraX, retornaAncoraY, getchair

1.6. FishTank

Tipo: Classe

Função: Esta classe cria e desenha graficamente um aquário.

1.6.1. Parâmetros

ancora, size, hb, hX, hY, win

1.6.2. Métodos (aplicável para classes)

__init__, cont, draw, undraw, retornaAncoraX, retornaAncoraY, getfishtank

1.7. Mess

Tipo: Classe

Função: Esta classe cria e desenha graficamente os pontos de maior sujeira.

FUNDAMENTOS DE PROGRAMAÇÃO

1.7.1. Parâmetros

ancora, size, win, dx, dy

1.7.2. Métodos

__init__, draw, undraw, getX, getY, getCenter, getmess, move

1.8. Dock

Tipo: Classe

Função: Esta classe cria e desenha graficamente as zonas de carregamento.

1.8.1. Parâmetros

ancora, width, height, win

1.8.2. Métodos (aplicável para classes)

__init__, draw, , getX, getY, getXY, getWidth, getHeight

1.9. Menu/Submenu

Tipo: Classe

Função: Criar um menu gráfico que permite ao utilizador seleccionar a implementação desejada.

1.9.1. Métodos

__init__, clicar, main

1.10. Button/MenuButton

Tipo: Classe

Função: Criar e desenhar graficamente um botão de modo a permitir a interação com o programa.

1.10.1. Métodos

__init__, clicar, main

Programa Principal

1.11. vetor

Tipo: Função

Função: Calcula o vetor unitário entre dois pontos no plano.

1.11.1. Entradas

aX, aY, bX, bY

1.11.2. Saídas (aplicável para funções)

vetoruni

1.12. exitbutton

Tipo: Função

Função: Cria e retorna um botão de saída com a posição, largura, altura e cor especificadas.

1.12.1. Entradas

win, ancora, width, height

1.12.2. Saídas

exitButton

1.13. invalidlocal

Tipo: Função

Função: Verifica se o ponto clicado é inválido, com base na lista de obstáculos e nas dimensões da janela.

1.13.1. Entradas

obstacles, clique, wradius, width, height

1.13.2. Saídas

True/False

1.14. choosedock

Tipo: Função

Função: Escolhe o local do dock mais próximo para o robô regressar.

1.14.1. Entradas

waiter,dock1,dock2

1.14.2. Saídas

p

1.15. Limpeza

Tipo: Função

Função: Realiza uma animação que simula o movimento de limpeza do robô até uma posição específica

1.15.1. Entradas

waiter, wradius, l, speed, move

1.16. gocharge

Tipo: Função

Função: Move o robô até à dock mais conveniente para recarregar sua bateria.

1.16.1. Entradas

dock1,dock2,waiter,speed,charge,traject

1.17. searchmess

Tipo: Função

FUNDAMENTOS DE PROGRAMAÇÃO

Função: Procura sujidades na sala para realizar a sua limpeza.

1.17.1. Entradas

sujidades, waiter, traject, reach, wradius, obstaculos, update, move

1.18. menu/filemenu/aleatmenu

Tipo: Função

Função: projetam um menu gráfico que executa as ações correspondentes às opções escolhidas.

1.19. aleatpoint

Tipo: Função

Função: Gera obstáculos aleatórios na sala

1.19.1. Entradas

win, wradius, width, height, buttonwidth, buttonheight

1.19.2. Saídas

formas